

1 Supplementary Methods

Class-wise Weighted Oversampling

To mitigate the severe imbalance between “easy” and “hard” patches ($\approx 50:1$), we adopt the customized weighted-oversampling strategy described in (Supplementary References 1). For a training dataset of N_{Train} samples, the sampling weight for each training sample i was calculated based on its class (“easy” or “hard” image patches), as outlined below:

$$w_{\text{Class}}(i, \gamma_s) = \frac{N_{\text{Train}}}{\gamma_s \left(\sum_{j \in [1, N_{\text{Train}}] | \ell_j = \ell_i} 1 \right) + (1 - \gamma_s) N_{\text{Train}}} \quad (1)$$

As denoted, $w_{\text{Class}}(i, \gamma_s)$ represents the sampling weight assigned to image patch i based on its class. Higher weights will be assigned to underrepresented classes, ensuring they are sampled more frequently during training. $\gamma_s \in [0, 1]$ is the oversampling factor that controls the strength of oversampling. When $\gamma_s = 0$, there is no oversampling, and all patches are sampled uniformly. When $\gamma_s = 1$, the maximum balancing is applied, and patches from underrepresented classes are given the highest weights. In this work, we followed the default setting of $\gamma_s = 0.85$ in (Supplementary References 1). $\sum_{j \in [1, N_{\text{Train}}] | \ell_j = \ell_i} 1$, this sum counts the number of patches in the training dataset that belong to the same class as patch i . Specifically, ℓ_j and ℓ_i represent the class labels of patches j and i , respectively. The numerator N_{Train} ensures that the weights are normalized based on the total number of training patches. The expression $\gamma_s \left(\sum_{j \in [1, N_{\text{Train}}] | \ell_j = \ell_i} 1 \right) + (1 - \gamma_s) N_{\text{Train}}$ controls the balance between classes. When γ_s is large, equation (1) gives more weight to underrepresented classes (with fewer samples), while for smaller γ_s , equation(1) leads to more uniform sampling. In summary, this oversampling approach ensures that human annotated samples, such as those with challenging nuclei or lower image quality, are sampled more frequently during training, which helps improve the model’s ability to generalize across all cell nuclei in the kidney.

2 Supplementary Experiments

2.1 Evaluation Metrics

To assess the instance segmentation performance of the foundation models after fine-tuning, we used Recall (equation(2)), Precision (equation(3)), and F1 score (equation(4)) as evaluation metrics.

Recall measures the ability of the model to correctly identify all nuclei instances in an image patch.

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}} = \frac{TP}{TP + FN} \quad (2)$$

Precision measures the proportion of correctly identified nuclei instances among all the instances predicted by the model.

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}} = \frac{TP}{TP + FP} \quad (3)$$

F1 score is calculated as the harmonic mean of precision and recall, providing a single metric that balances both by weighting their average.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \times TP}{2 \times TP + FP + FN} \quad (4)$$

Moreover, the Intersection over Union (IoU) threshold is set to 0.5 to consider a detection as a True Positive (TP). $|TP|$, $|FP|$, and $|FN|$ are the number of True Positives, False Positives, and False Negatives, respectively.

- **True Positives (TP):** Matched pairs of nuclei, i.e., correctly detected nuclei.
- **False Positives (FP):** Unmatched predicted nuclei, i.e., predicted nuclei without matching ground truth (GT) nuclei.
- **False Negatives (FN):** Unmatched GT nuclei, i.e., GT nuclei without matching predicted nuclei.

2.2 Implementation Details

To maintain consistency in experimental settings and enable better comparison of fine-tuned performance, we applied the same oversampling factor $\gamma_s = 0.85$ across all experiments that used both “easy” and “hard” image patches. Due to the imbalance between “easy” and “hard” image patches, the number of training epochs was set to 50 for all fine-tuning experiments involving “easy” image patches, while experiments using only “hard” image

patches were trained for 25 epochs. The batch size was consistently set to 16 across all experiments.

Cellpose Training: For training Cellpose using our true color images, we first converted the images into the format required for training the Cellpose nucleus model. Each image was converted into a 2-band format, where the first band represents the grayscale image and the second band consists entirely of zeros. This follows the default Cellpose nucleus model inference format, where the first channel is used for nuclei segmentation. Due to different loss convergence patterns in training Cellpose, the base learning rate was set to 0.1 for fine-tuning with only “hard” image patches and 0.001 for fine-tuning that includes “easy” image patches. If the training is not stable, set the number of epochs to 100 for fine-tuning experiments involving “easy” image patches and 50 for fine-tuning experiments with “hard” image patches only. Other experimental settings are unchanged according to (Supplementary Reference 2).

StarDist Training: The learning rate was maintained at 0.0003 (default), with all other settings unchanged as in (Supplementary Reference 3). In this work, the fine-tuned StarDist model can be converted and loaded into QuPath extension format for easy validation and labeling purposes.

CellViT Training: The base learning rate was set to 0.0003, with a scheduling factor of 0.85 to gradually reduce the learning rate during training. HoVer-Net post-processing with equal weighting on each loss objective (excluding nuclei classification and tissue classification) was applied during training. The pretrained ViT₂₅₆ encoder and decoder were trained end-to-end, with no layers frozen. All other hyperparameters remained the same as in the original setting (Supplementary Reference 1).

Supplementary References

F. H^orst, M. Rempe, L. Heine, et al., “Cellvit: Vision transformers for precise cell segmentation and classification,” *Medical Image Analysis*, 103143 (2024).

C. Stringer, M. Michaelos, and M. Pachitariu, “Cellpose: a generalist algorithm for cellular segmentation,” *bioRxiv* (2020)

M. Weigert and U. Schmidt, “Nuclei instance segmentation and classification in histopathology images with stardist,” in *2022 IEEE International Symposium on Biomedical Imaging Challenges (ISBIC)*, 1–4, IEEE (2022)

3 Supplementary Figures S1, S2, and S3

More examples of Qualitative Results of Enhanced Kidney Cell Nuclei Instance Segmentation are provided in Fig. S1, S2, and S3.

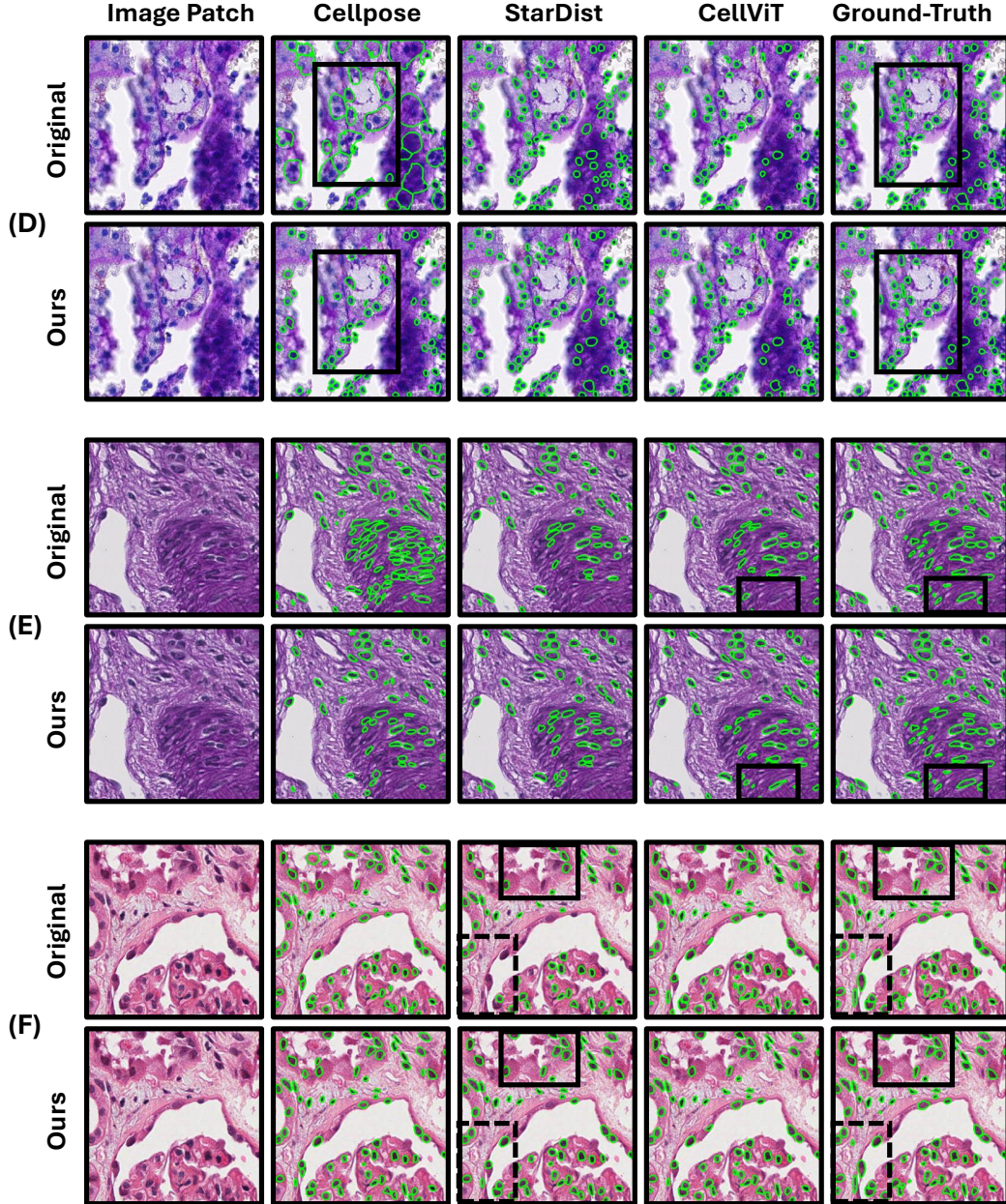


Fig. S1: Qualitative Results of Enhanced Kidney Cell Nuclei Instance Segmentation. The predicted nuclei and ground truth are represented as overlaid green contours on the image patch, with areas of improvement highlighted by rectangles.

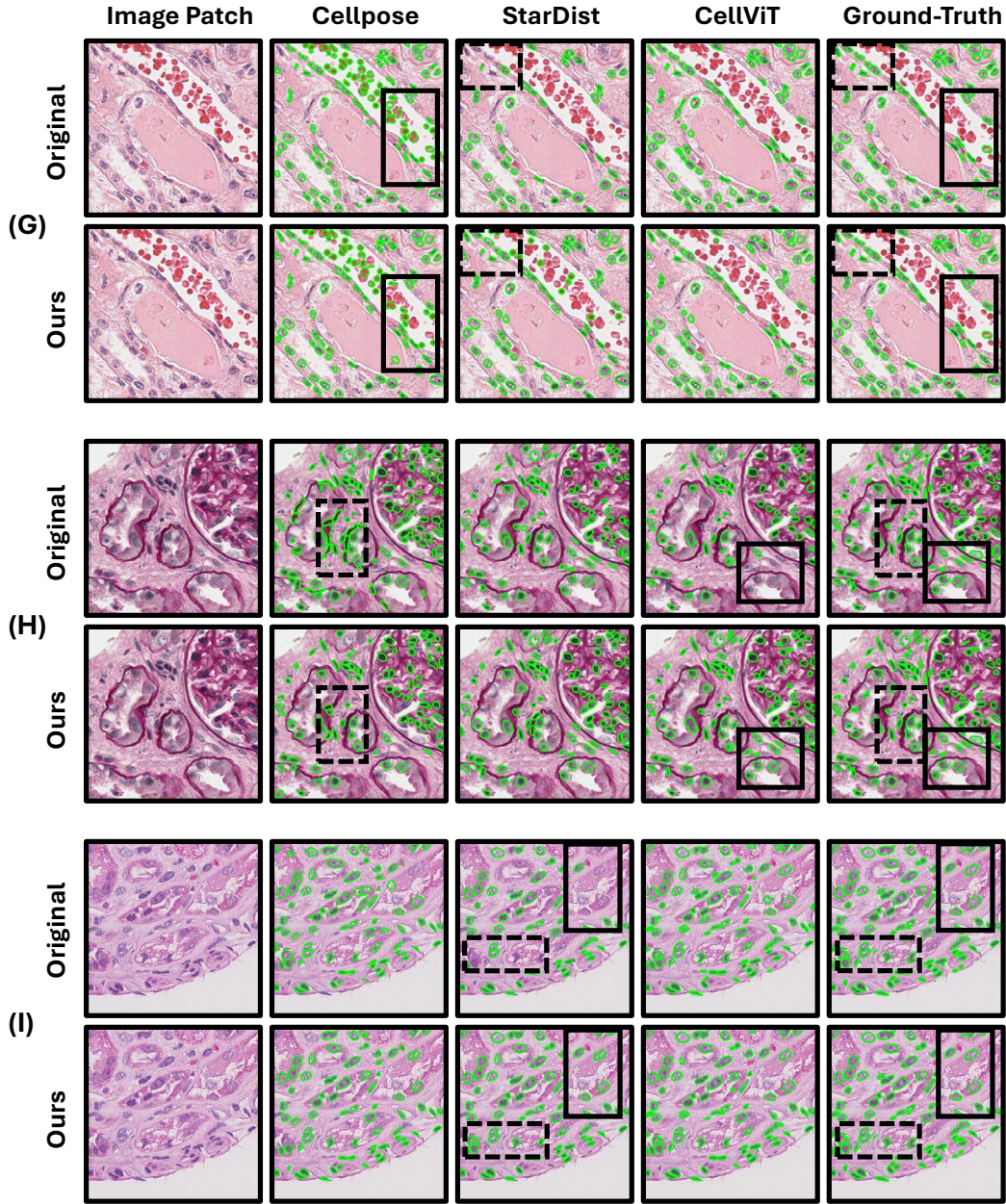


Fig. S2: Qualitative Results of Enhanced Kidney Cell Nuclei Instance Segmentation. The predicted nuclei and ground truth are represented as overlaid green contours on the image patch, with areas of improvement highlighted by rectangles.

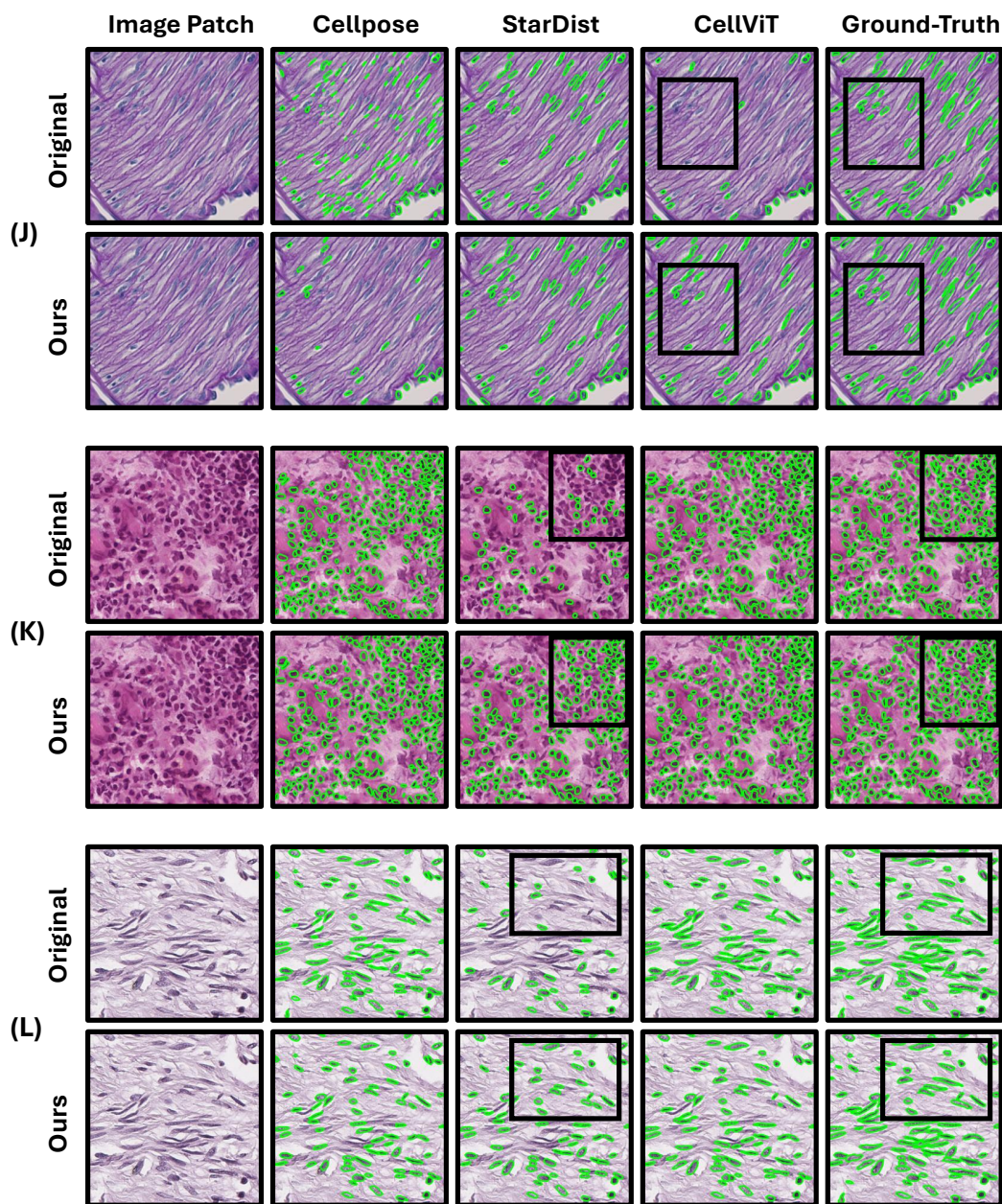


Fig. S3: Qualitative Results of Enhanced Kidney Cell Nuclei Instance Segmentation. The predicted nuclei and ground truth are represented as overlaid green contours on the image patch, with areas of improvement highlighted by rectangles.

4 Supplementary Table S1, S2

Model	Inter-rater agreement (%) $\uparrow$
Cellpose	92.2
StarDist	92.6
CellViT	94.3

Table S1: **Inter-Rater Agreement.** We randomly selected 300 images for an inter-rater reliability test. For each model, we report the percentages (%) of samples where the two student raters agreed.

Annotation Strategies	Cellpose	StarDist	CellViT	Average Performance
Baseline	0.6748 ± 0.032	0.7380 ± 0.024	0.7838 ± 0.021	0.7322 ± 0.026
Easy 100%	0.7326 ± 0.011	0.7943 ± 0.011	0.7780 ± 0.015	0.7683 ± 0.012
Hard 100%	0.5792 ± 0.015	0.8276 ± 0.025	0.7860 ± 0.022	0.7309 ± 0.021
Easy 100% + Hard 100%	0.7325 ± 0.009	0.8181 ± 0.013	0.7939 ± 0.016	0.7815 ± 0.013

Table S2: Comparison of model performance (F1-score) under different annotation strategies using 5-fold cross-validation, where each value represents the mean $\pm$ std. Average Performance calculates the mean per row, which shows that the combination of both annotation types yields the best results.